

ISLAMIC UNIVERSITY OF TECHNOLOGY

Board Bazar, Gazipur, Dhaka.



Artificial Intelligence Lab: CSE 4618 **Lab 6 - Reinforcement Learning**

Written by - Namisa Najah Raisa Student ID: 210042112

Sixth Semester
Department of Computer Science and Engineering
BSc. in Software Engineering

September 4, 2025

Contents

1	Question 6 : Q-Learning	3
1.1	Problem Analysis	3
1.2	Solution	3
1.2.1	init	7
1.2.2	getQValue(self, state, action)	8
1.2.3	computeValueFromQValues(self, state)	8
1.2.4	computeActionFromQValues(self, state)	8
1.2.5	update(self, state, action, nextState, reward)	9
1.3	Findings	9
1.4	Challenges	9

1 Question 6 : Q-Learning

1.1 Problem Analysis

In this lab, the task was to implement a **Q-learning agent** that learns optimal behavior through trial-and-error interactions with the environment, rather than relying on a pre-defined MDP model as in value iteration. Specifically, the Q-learning agent was required to update its knowledge of the environment using the **update(state, action, nextState, reward)** method, and derive decision-making strategies by implementing the following core methods in **qlearningAgents.py**:

- **getQValue** – return the Q-value for a given state-action pair.
- **computeValueFromQValues** – compute the maximum Q-value across all possible actions in a state.
- **computeActionFromQValues** – determine the optimal action from a given state based on Q-values.
- **update** – adjust Q-values using the Q-learning update rule from experience tuples.

1.2 Solution

The task asked only for *update*, *computeValueFromQValues*, *getQValue*, and *computeActionFromQValues* methods.

```

class QLearningAgent(ReinforcementAgent):
    """
    Q-Learning Agent

    Functions you should fill in:
        - computeValueFromQValues
        - computeActionFromQValues
        - getQValue
        - getAction
        - update

    Instance variables you have access to
        - self.epsilon (exploration prob)
        - self.alpha (learning rate)
        - self.discount (discount rate)

    Functions you should use
        - self.getLegalActions(state)
          which returns legal actions for a state
    """
    def __init__(self, **args):
        "You can initialize Q-values here..."
        ReinforcementAgent.__init__(self, **args)

        *** YOUR CODE HERE ***
        # 210042112
        self.qValues = util.Counter()

```

Figure 1: init function

```

def getQValue(self, state, action):
    """
        Returns  $Q(state, action)$ 
        Should return 0.0 if we have never seen a state
        or the  $Q$  node value otherwise
    """
    """* YOUR CODE HERE """
    # util.raiseNotDefined()
    # 210042112
    return self.qValues[(state, action)]

```

Figure 2: getQvalue function

```

def computeValueFromQValues(self, state):
    # get the max from all the actions of a state
    """
        Returns  $\max_{action} Q(state, action)$ 
        where the max is over legal actions. Note that if
        there are no legal actions, which is the case at the
        terminal state, you should return a value of 0.0.
    """
    """* YOUR CODE HERE """
    # util.raiseNotDefined()
    # 210042112
    legalActions = self.getLegalActions(state)
    if not legalActions:
        return 0.0
    return max([self.getQValue(state, action) for action in legalActions])

```

Figure 3: computeValueFromQValues function

```

def computeActionFromQValues(self, state):
    """
        Compute the best action to take in a state. Note that if there
        are no legal actions, which is the case at the terminal state,
        you should return None.
    """
    """ YOUR CODE HERE """
    # 210042112
    #util.raiseNotDefined()

    legalActions = self.getLegalActions(state)
    if not legalActions:
        return None

    best_value = self.computeValueFromQValues(state)
    best_actions = [a for a in legalActions if self.getQValue(state, a) == best_value]
    return random.choice(best_actions)

```

Figure 4: computeActionFromQValues function

```

def update(self, state, action, nextState, reward):
    """
        The parent class calls this to observe a
        state = action => nextState and reward transition.
        You should do your Q-Value update here

        NOTE: You should never call this function,
        it will be called on your behalf
    """
    """ YOUR CODE HERE """
    #util.raiseNotDefined()
    # 210042112
    difference = (reward + self.discount * self.computeValueFromQValues(nextState)) - self.getQValue(state, action)
    self.qValues[(state,action)] += self.alpha * difference

```

Figure 5: Update function

```
Question q6
=====

*** PASS: test_cases\q6\1-tinygrid.test
*** PASS: test_cases\q6\2-tinygrid-noisy.test
*** PASS: test_cases\q6\3-bridge.test
*** PASS: test_cases\q6\4-discountgrid.test

### Question q6: 4/4 ###

Finished at 23:54:37

Provisional grades
=====
Question q6: 4/4
-----
Total: 4/4
```

Figure 6: Scoring

I implemented a tabular Q-learning agent that stores Q-values in a dictionary-like structure, updates them using the standard temporal-difference rule, and selects greedy actions based on the current Q-table. The core idea is that the agent learns from experience tuples (s, a, s', r) and updates its estimate of $Q(s, a)$ towards the sample target.

1.2.1 init

I initialized the Q-value store as follows:

```
self.qValues = util.Counter()
```

I did this because `util.Counter()` behaves like a dictionary with default value 0.0 for unseen keys. This means querying a Q-value that has never been encountered returns 0.0 rather than raising a **KeyError**. Using this default simplifies early learning when most state-action pairs are uninitialized.

1.2.2 getQValue(self, state, action)

I implemented the Q-value accessor:

```
return self.qValues[(state, action)]
```

because the Q-table is keyed by the tuple (state, action). With **util.Counter()**, unseen pairs return 0.0, matching the Q-learning assumption that unknown Q-values start at zero.

1.2.3 computeValueFromQValues(self, state)

I computed the state value $V(s) = \max_a Q(s, a)$ like this:

- I retrieved legal actions via **self.getLegalActions(state)**.
- If there are no legal actions (i.e., terminal state), I returned 0.0.
- Otherwise I returned the maximum Q-value across actions: $\max_a Q(s, a)$.

In code:

```
legalActions = self.getLegalActions(state)
if not legalActions:
    return 0.0
return max([self.getQValue(state, action) for action in legalActions])
```

I did this because the Q-learning update bootstraps on the best future action: the update needs $\max_{a'} Q(s', a')$ as the estimate of optimal future return. Returning 0.0 for terminal states ensures updates for terminal transitions use only the immediate reward (no bootstrapped future value), which is correct.

1.2.4 computeActionFromQValues(self, state)

I implemented greedy action selection with uniform random tie-breaking:

- If no legal actions exist, return **None**.
- Compute $\text{best_value} = \max_a Q(s, a)$.
- Collect all actions achieving best_value .
- Return a uniformly random choice among the best actions.

In code:

```
legalActions = self.getLegalActions(state)
if not legalActions:
    return None
best_value = self.computeValueFromQValues(state)
best_actions = [a for a in legalActions if
                 self.getQValue(state, a) == best_value]
return random.choice(best_actions)
```


I did this because the greedy policy picks an action that maximizes $Q(s,a)$. When multiple actions tie for the maximum, uniformly random tie-breaking avoids deterministic bias and encourages exploration among equally good options.

1.2.5 `update(self, state, action, nextState, reward)`

I implemented the standard Q-learning temporal-difference update:

```
difference = (reward + self.discount * self.computeValueFromQValues(nextState)) -
self.getQValue(state, action)

self.qValues[(state, action)] += self.alpha * difference
```

In mathematical form:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left(r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right),$$

where:

- r is the observed reward (**reward**),
- γ is the discount factor (**self.discount**),
- α is the learning rate (**self.alpha**),
- $\max_{a'} Q(s', a')$ is estimated by **computeValueFromQValues(nextState)**.

I did this because this is the textbook Q-learning rule. The term inside parentheses is the temporal-difference error δ , and updating $Q(s, a)$ by $\alpha\delta$ moves the estimate toward the observed sample target. Repeating this across many transitions and episodes enables the agent to converge to optimal Q-values under standard conditions.

1.3 Findings

While writing the Q-learning agent, I found that using **util.Counter** was very helpful because it set unseen Q-values to 0 automatically. For terminal states, I had to return 0.0; otherwise, the updates did not work properly. Adding random tie-breaking when actions had the same value was important since it kept the agent from always picking the same move. Overall, the code worked as expected, and I understood better how Q-values are updated through experience.

1.4 Challenges

One challenge I faced was figuring out how to handle terminal states, since mistaking to return 0.0 caused wrong updates. Another challenge was understanding the update formula and making sure I used the right values in the equation. At first, I also struggled with choosing actions when there were ties, but using random choice solved that problem as it was instructed in the lab manual.